

Data Structures

Data structures

Joseph Mhango

2024-09-13

Objectives

By the end of this section, you should be able to:

- Identify and describe basic data types in R.
 - Understand how factors represent categorical data.
 - Use `class()` and conversion functions to change variable types.
 - Work with vectors, matrices, and lists.
 - Apply best practices in naming and managing R objects.
-

Basic data types in R

R works with different kinds of **data objects**, and each object has an associated **type**. These types determine what operations are possible and how the data is treated during analysis. All the data you create lives in the **Global Environment**, and it's important to know what kind of data you're working with.

R is pretty good at guessing what type your data should be – but this can sometimes backfire. Think of R as a *passive-aggressive butler*: it tries to help, but doesn't always tell you when it's done something –helpful. So, you should learn to check and control data types explicitly.

Basic primitive types in R

Here are the most common **primitive data types**:

```
a <- 1      # numeric  
mode(a)
```

```
[1] "numeric"
```

```
a <- "1"    # character  
mode(a)
```

```
[1] "character"
```

```
a <- '1'    # character  
mode(a)
```

```
[1] "character"
```

```
a <- TRUE   # logical  
mode(a)
```

```
[1] "logical"
```

```
a <- 1i     # complex  
mode(a)
```

```
[1] "complex"
```

These types include: - **numeric**: Numbers like 1, 3.14 - **character**: Strings like “apple” - **logical**: TRUE or FALSE - **complex**: Numbers with imaginary parts like 1i

Special values in R

There are a few special types of values that youâ€™ll come across often:

- NULL means an undefined or empty object (a missing vector). Use `is.null()` to test it.
- NA stands for *Not Available* and represents missing data. Check with `is.na()`.
- Inf and -Inf mean positive or negative infinity (e.g., from $1/0$). Use `is.infinite()`.
- NaN means *Not a Number*, e.g. $0/0$. Use `is.nan()`.

```
my_list <- list(a = 1, b = 2)
my_list <- my_list$c # Element 'c' doesn't exist, Output: NULL

na_addition <- 5 + NA
print(na_addition) # Output: NA
```

```
[1] NA
```

Naming objects in R

There are some general rules and conventions for naming objects in R:

- Names must start with a letter.
- Names can include letters, numbers, underscores (`_`), and dots (`.`).
- Avoid special characters like `@`, `%`, `#`, etc.
- Avoid spaces â€“ use underscores or camelCase instead.
- Use names that are short, descriptive, and consistent.

```
x1 <- 5

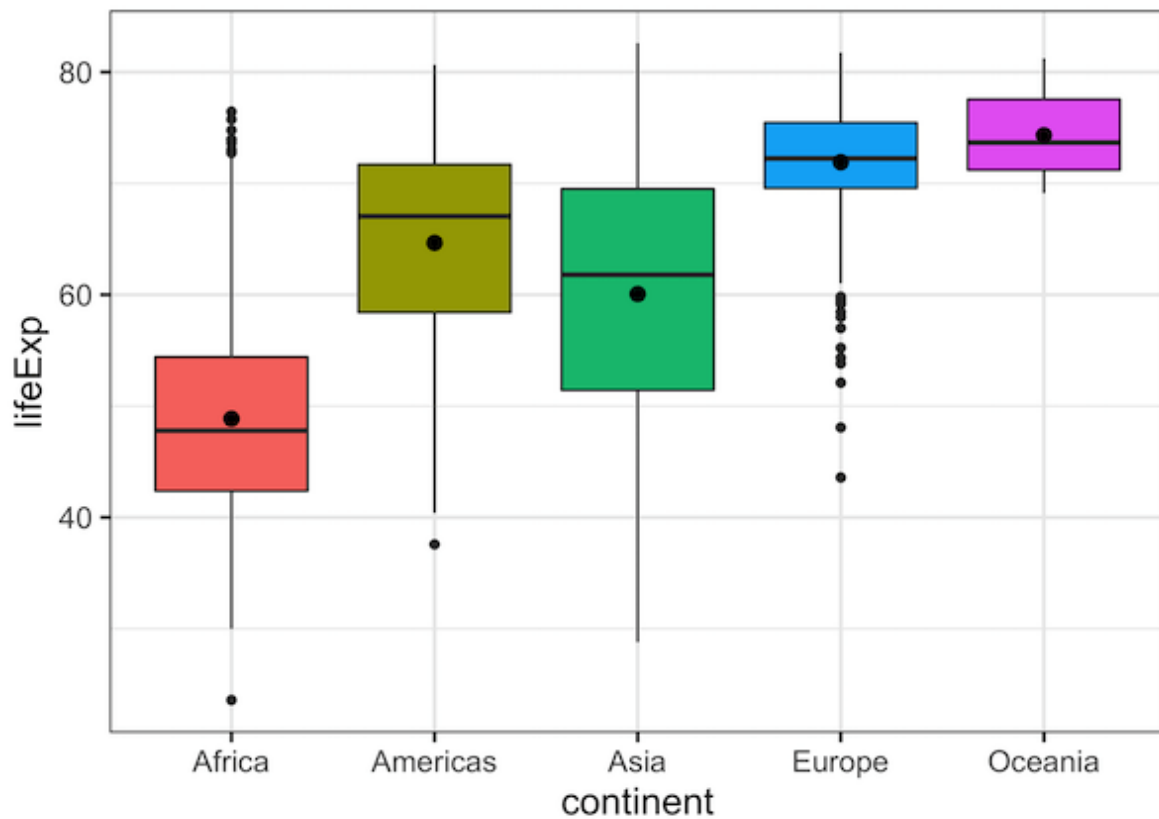
x2 <- "It was a dark and stormy night"

# harder to read and maintain:
my_variable_9283467 <- 1
```

Data with factors

Factors are how R represents **categorical data**. This is useful when you have data that falls into named categories, such as treatment groups, soil types, or crop varieties.

Let's say you want to analyse categorical data. Sometimes this is called a *factor*.



There are two types of factors:

- **Unordered (nominal):** No implied order among categories.
 - Example: `cow_names <- factor(c("Martha", "Ruth", "Edith"))`
- **Ordered:** There is a logical order.
 - Example: `dose <- factor(c("Control", "Half", "Full"), ordered = TRUE)`

Using `class()` and converting variables

You can check the type of an object with `class()` or `mode()`, and convert between types using `as.*` functions.

```
# non-ordered factor
variety <- c("short", "short", "short",
            "early", "early", "early",
            "hybrid", "hybrid", "hybrid")

factor_variety <- factor(variety) # creates an unordered factor
class(factor_variety)
```

```
[1] "factor"
```

Vector and Matrix fun

These are the fundamental data containers in R. They organise values in structured ways:

- **Vector:** One-dimensional, holds values of the same type. Indexed as `my_vec[i]`
- **Matrix:** Two-dimensional, also holds only one type. Indexed as `my_mat[i, j]`
- **Array:** Extends matrix to 3+ dimensions, indexed as `my_array[i, j, k]`

```
# Vectors
myvec1 <- c(1,2,3,4,5)
myvec1
```

```
[1] 1 2 3 4 5
```

```
class(myvec1)
```

```
[1] "numeric"
```

```
myvec2 <- as.character(myvec1)
```

```
# Matrices
vec1 <- 1:16
vec1
```

```
[1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
```

```
mat1 <- matrix(data = vec1,
               ncol = 4,
               byrow = FALSE)
mat1
```

```
      [,1] [,2] [,3] [,4]
[1,]     1     5     9    13
[2,]     2     6    10    14
[3,]     3     7    11    15
[4,]     4     8    12    16
```

Lists

Unlike vectors and matrices, **lists** can contain elements of different types. This makes them ideal for storing heterogeneous information such as model outputs, metadata, or complex data structures.

```
# Creating a list
my_list <- list(name = "John", age = 25, scores = c(90, 85, 88))
print(my_list)
```

```
$name
[1] "John"
```

```
$age
[1] 25
```

```
$scores
[1] 90 85 88
```

```
# Accessing elements
my_list$name      # by name
```

```
[1] "John"
```

```
my_list[["age"]]  # by key
```

```
[1] 25
```

```
my_list[[3]][1]   # first score
```

```
[1] 90
```

Each element can itself be a vector, matrix, another list, or a data frame.

Summary

- Data in R is stored as objects, and each has a **type** and **structure**.
 - Learn to inspect and convert types (`mode()`, `class()`, `as.character()` etc.).
 - Vectors and matrices are efficient for numeric data, while lists and data frames offer more flexibility.
 - Naming and structuring your data properly will save you headaches down the line.
 - Factors help you manage categorical data and are essential for plotting and modelling.
- ““

Practice Exercises

